



Módulo 2 · Repaso y práctica de lo visto en clase

Refactorización y Mejora de Código Existente

Ejercicio de Clase

Nombre del alumno:

ANGEL DAVID ECHENIQUE VILLANUEVA

Fecha:

25/02/26

Instrucciones

Responde las preguntas con lo que hemos visto en clase. Si tienes dudas, puedes consultar tus apuntes. No hay que escribir código, ¡solo demostrar que entendiste!



BLOQUE
E 1

¿Qué es la refactorización?

Hemos visto que refactorizar no es lo mismo que reescribir un programa desde cero, ni arreglar bugs. Es algo más concreto y con un objetivo muy claro.

Ejercicio 1.1 Define con tus palabras

Es mejorar el código mal estructurado, sucio o complejo a código limpio, legible y mantenible para el futuro. Siempre y cuando no modifiquemos el comportamiento externo.

Explica con tus propias palabras qué significa la definición anterior. ¿Qué quiere decir "sin modificar su comportamiento externo"?

Significa que cambiamos como está escrito el código por dentro, mejoramos la organización, claridad y estructura, pero el usuario no ve ningún cambio en lo que hace el programa.

Ejercicio 1.2 Objetivos y beneficios

En clase vimos 7 objetivos principales de la refactorización. Escríbelos:

1	Mejoramos la legibilidad del código.
2	Quitamos código duplicado.
3	Facilidad para el mantenimiento.
4	Reducimos la complejidad de código.
5	Aplicamos patrones de diseños.
6	Optimizamos el rendimiento.
7	Aumentamos la calidad del software.

¿Y cuáles son los 4 beneficios principales? Escríbelos:

1	Menos errores en el futuro.
2	Código limpio.
3	Un proyecto que puede seguir escalando.
4	Facilidad para testear.

Ejercicio 1.3 Verdadero o Falso

Marca si cada afirmación es Verdadera (V) o Falsa (F):

V / F	Afirmación	Nº
☐ F	Refactorizar significa cambiar lo que hace el programa para mejorarlo.	1
☐ V	El objetivo principal de refactorizar es que el código sea más fácil de leer y mantener.	2
☐ F	La refactorización solo tiene sentido en proyectos nuevos, no en código legacy.	3
☐ V	Después de refactorizar bien, el programa debería seguir haciendo exactamente lo mismo.	4
☐ V	Refactorizar puede ayudar a reducir el número de errores en el futuro.	5



En clase vimos 6 tipos de refactorización que la IA puede realizar según el objetivo que tengamos.

Ejercicio 2.1 Une cada tipo con su descripción

Une cada tipo de refactorización con lo que hace:

1. Simplificación	→ Convierte código antiguo a versiones modernas (ej. jQuery → React)
2. Modularización	→ Renombra variables y funciones para que sean más descriptivas
3. Mejora de nomenclatura	→ Reduce complejidad ciclomática, elimina condicionales anidados
4. Optimización	→ Aplica Strategy, Factory, Singleton u otros patrones al código
5. Aplicación de patrones	→ Mejora la velocidad y uso de memoria, reduce $O(n^2)$ a $O(n)$
6. Modernización de código legacy	→ Divide funciones grandes en piezas pequeñas y reutilizables

Consejo

Pista: las descripciones están mezcladas.

Léelas todas antes de unir para no confundirte con tipos parecidos.

Ejercicio 2.2 El prompt típico de cada tipo

En clase vimos un prompt típico para cada tipo de refactorización. Escribe el nombre del tipo que corresponde a cada prompt:

Prompt típico	Tipo de refactorización
"Simplifica esta función eliminando anidación innecesaria."	SIMPLIFICACION
"Divide esta función de 50 líneas en funciones más pequeñas con responsabilidad única."	MODULACION

"Renombra variables y funciones siguiendo convenciones de Clean Code."	MEJORAS DE NOMENCLATURA
"Optimiza este algoritmo para mejorar su complejidad temporal."	OPTIMIZACION
"Refactoriza este código usando el patrón Strategy para eliminar condicionales."	APLICACION DE PATRONES DE DISEÑOS
"Moderniza este código jQuery a React Hooks."	MODERNIZACION DE CODIGO LEGACY

Ejercicio 2.3 Ejemplos de mejora de nomenclatura

En clase vimos estos ejemplos de nombres malos que se mejoran. Completa la columna de la derecha con el nombre mejorado que recuerdes del módulo, o propón uno tú mismo que siga las convenciones de Clean Code:

Nombre original (malo)	Nombre mejorado (claro)
calc()	calcularTotal()
d, p	días / pedidos
t	total
fn1()	procesarPedido()
lst	listaProductos
res	resultado



Workflow profesional de refactorización con IA

BLOQUE 3

Vimos que refactorizar con IA sigue un proceso de 5 pasos. No se hace todo de golpe, sino de forma ordenada e incremental.

Ejercicio 3.1 Ordena los pasos del workflow

Los 5 pasos del workflow aparecen desordenados. Escribe el número de orden correcto (del 1 al 5) en la columna de la izquierda:

Orden	Paso
<u>5</u>	Documentación — La IA actualiza comentarios y documentación del código
<u>2</u>	Plan de refactorización — La IA propone un plan ordenado por impacto y dificultad
<u>1</u>	Análisis inicial — La IA identifica code smells, duplicidades y riesgos
<u>4</u>	Validación — La IA genera tests para asegurar que el comportamiento no cambió
<u>3</u>	Ejecución incremental — Se refactoriza por partes, no todo a la vez

Ejercicio 3.2 ¿Qué hace la IA en cada paso?

Completa la tabla indicando qué hace la IA y qué hace el programador en cada fase del workflow:

Paso	¿Qué hace la IA?	¿Qué hace el programador?
Análisis inicial	La IA identifica code smells, duplicidades y riesgos	Nosotros revisamos ese análisis que hace ella para comprobar si es correcto luego decidiremos que problemas son prioritarios.
Plan de refactorización	La IA propone un plan ordenado por impacto y dificultad	Nos encargamos de aceptar, modificar o reorganizar el plan según el proyecto que tengamos delante.
Ejecución incremental	Se refactoriza por partes, no todo a la vez	Supervisamos los cambios hecho por ella en el código.
Validación	La IA genera tests para asegurar que el comportamiento no cambió	Ejecutamos los test, revisamos los resultados y verificamos que el comportamiento siga siendo correcto.
Documentación	La IA actualiza comentarios y documentación del código	Revisamos nuestra documentación a ver que ha cambiado y si sigue siendo la correcta.

Ejercicio 3.3 Los prompts del workflow

En clase vimos un prompt concreto para cada paso. Escribe el prompt que usarías en cada uno de estos pasos:

Paso 1 — Análisis inicial:

 Tu prompt:

Analizame este código y detectame code smell, duplicaciones y descríbeme los problemas que tengo.

Paso 4 — Validación:

 Tu prompt:

He refactorizado mi código y necesito que me generes test unitarios para confirmar que mi código se sigue comportando igual.



BLOQUE 4

Buenas prácticas y errores comunes

En clase vimos qué hay que hacer bien cuando refactorizamos con IA, y también los errores más frecuentes que cometen los programadores.

Ejercicio 4.1 ¿Buena práctica o error común?

Clasifica cada afirmación escribiendo BP (Buena Práctica) o EC (Error Común):

BP / EC	Afirmación
Bp	Tener tests escritos antes de empezar a refactorizar.
Ec	Aceptar todos los cambios que propone la IA sin revisarlos.
Bp	Refactorizar en incrementos pequeños, validando cada cambio.
Ec	Intentar refactorizar todo el proyecto de una sola vez.
Bp	Medir métricas de calidad antes y después de refactorizar.
Ec	No revisar la seguridad del código generado por la IA.
Bp	Documentar las decisiones de refactorización tomadas.
Ec	No ejecutar tests después de cada cambio incremental.

Ejercicio 4.2 Reflexión

En clase vimos que uno de los errores más graves es aceptar los cambios de la IA sin entenderlos. ¿Por qué crees que esto es un problema? Razona tu respuesta con al menos dos ideas:

Porque tendríamos un código que nosotros ni sabríamos que hace o como se comporta a hora, nosotros mismo ni sabríamos que cambiar si preguntarle a ella, osea el código ahora no es de nosotros es de ella. Porque ella entiendo y sabe lo que cambio, Nosotros aun no sabemos ni que hizo.

Ahora imagina que tienes una app funcional y decides refactorizar el código y entonces procede hacerlo y tu/yo no revisa solo copias y pegas y corres la app siguen funcionando igual pero justo debajo tienes un archivo con un código que se roba toda tu información y tu ni te enteras por espabilao.



BLOQU
E 5

Pregunta de síntesis

Un compañero tuyo recibe este código y le pide a la IA que lo mejore usando este prompt:

"Arregla este código."

1. ¿Qué tipo de refactorización necesita este código? ¿Cómo lo sabrías sin verlo? ¿Qué le preguntarías a tu compañero para saberlo?

Es un prompt inncesario y muy inutil.

Necesitamos saber sobre que es el codigo y contexto del mismo.

Le preguntaria si el codigo funciona primero, luego le preguntaria que busca con la refactorizacion para ver si eso es lo que necesita o si necesita otra cosa y el va por el camino equivocado.

2. Reescribe el prompt aplicando al menos 3 tipos de refactorización vistos en clase:

✓ **Mi prompt mejorado:**

Quiero que me lo analises y busques duplicacion, problemas de legibilidad de cogido.

Y aplicame simplificacion de condicionales, mejoras de nomenclatura siguiendo el clean code y optimizacion si detectas algun algoritmo ineficiente.

3. ¿Qué paso del workflow harías ANTES de ejecutar ese prompt? ¿Y DESPUÉS? Justifica por qué.

Indagar en el codigo en buscar de problemas serios, si tenemos riesgo al tocar algo porque no puedo ir a refactorizar algo que no se que problemas tiene.

Luego nos tocaria verificar mediante test que todo este pasando bien y que nuestro codigo sigue haciendo lo mismo que antes sin romper nada.
